

Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет
«Высшая школа экономики»

На правах рукописи

Соколов Павел Павлович

**ПРАКТИЧНЫЕ ЗАВИСИМЫЕ ТИПЫ: ВВЕДЕНИЕ ЛЕГКОВЕСНЫХ
МЕТОДОВ ФОРМАЛЬНОЙ ВЕРИФИКАЦИИ В МЕЙНСТРИМНОЕ
ПРОГРАММИРОВАНИЕ**

ДИССЕРТАЦИЯ
на соискание ученой степени
кандидата компьютерных наук

Научный руководитель:
доктор физико-математических наук,
профессор Кузнецов Степан Львович

Москва — 2027

Оглавление

Введение	3
Актуальность исследования	3
Степень разработанности исследовательской проблемы	3
Объект и предмет исследования	4
Цель и задачи исследования	5
Методология и методы исследования	5
Научная новизна исследования	5
Практическая значимость работы	6
Степень достоверности и апробация результатов	6
1 Резюме представленных к защите публикаций	7
Заключение	8
Основные результаты исследования	8
Общие выводы	8
Перспективы дальнейшей разработки	8
Основные публикации соискателя	8
A Список сокращений и условных обозначений	9
B Словарь терминов	10
C Список литературы	11
D Список иллюстрированного материала	12
E Приложения	13

Введение

Актуальность исследования

Подавляющее большинство используемых сегодня языков программирования не подходит для написания корректного по построению кода. В случае же, когда требуется удостовериться в корректном исполнении программы (исполнение без ошибок, соответствие спецификации и так далее), программисту необходимо настраивать дополнительные инструменты: статические анализаторы, автоматическое тестирование, средства проверки модели и так далее. К сожалению, ни одно из используемых средств не решает задачу полностью: какие-то из них не гарантируют корректности¹, другие имеют (сверх)экспоненциальные асимптотики исполнения и не могут быть использованы для полного анализа проверяемых систем². В конце концов, сама необходимость дополнительной настройки, коммерческое лицензирование и сложность использования мешают распространению подобных технологий.

Так, исчерпывающим решением было бы легковесное, бесплатное, удобное для использования, встроенное в стандартную поставку языка программирования средство формальной спецификации и формальной верификации программной системы; по определению, сформулированному Бенджамином К. Пирсом [14], таким средством является *система типов*. Приведём его здесь:

Система типов — это гибко управляемый синтаксический метод доказательства отсутствия в программе определенных видов поведения при помощи классификации выражений языка по разновидностям вычисляемых ими значений (*типам*).

Становится возможным поставить следующую *исследовательскую проблему*: Какой должна быть система типов языка программирования, чтобы

- Она позволяла легко формулировать и легко (в идеале — автоматически) доказывать свойства программ, интересные в мейнстримном программировании?
- Полученным языком было удобно пользоваться широкому кругу пользователей?

Степень разработанности исследовательской проблемы

К подтверждению гипотезы, что теорию типов можно использовать для формальной верификации компьютерных программ, логики и исследователи компьютерных наук приходили постепенно. Пожалуй, первым удачным экспериментом в этой области следует считать Исчисление Конструкций (Calculus of Constructions, CoC) [5] Тьерри Коканда, в

¹Здесь уместно процитировать Эдсгера В. Дейкстру: “... тестирование программы может весьма эффективно продемонстрировать наличие багов, но совершенно неприменимо, чтобы показать их отсутствие” [6].

²Однако, например, проверку моделей вполне успешно используют для верификации критических компонентов системы и рискованных оптимизаций. [8]

котором можно не только реализовывать алгоритмы в виде программ на некотором функциональном языке, но и доказывать их свойства в той же самой системе; практическая реализация этого исчисления сегодня известна как Rosq (ранее Coq) [11].

Эта замечательная особенность CoC возникла в результате принятия парадигмы *утверждения в качестве (некоторых) типов* [13]. В её рамках доказываемое утверждение φ кодируется в виде некоторого *типа* F , представимого в выбранной системе (так что язык типов должен быть достаточно выразительным, чтобы кодировать все утверждения, которые мы хотим доказать). Тогда задача доказательства $\vdash \varphi$ сводится к задаче построения выражения f , имеющего тип F ($\vdash f : F$); поиск такого выражения может производиться как вручную, так и с помощью автоматических средств³.

Однако следует иметь в виду, что Rosq и другие системы интерактивных доказательств в первую очередь являются *системами доказательств*, нежели *языками программирования*: как правило, скрипты для таких систем не запускаются в качестве самостоятельных программ, а только подаются на вход алгоритму проверки типов⁴. Значимыми исключениями являются языки Agda и Lean: в соответствующих компиляторах есть опции, позволяющие компилировать и запускать скрипты на этих языках.

Agda [9], как и язык, на котором она реализована (Haskell), в первую очередь является тестовой площадкой для различных расширений CoC: доступны опции для подключения коиндуктивных типов [2] и ограниченной рекурсии [4], кубических типов [12], логически противоречивого правила Type-in-Type и многих других. К сожалению, это делает ядро системы перегруженным и малоприменимым в критических ситуациях.

Система Lean [7] получила широкую известность ввиду прорывного по тогдашним меркам инструментария и распространения в среде математиков-энтузиастов. Кроме этого, архитектура системы типов и стандартных библиотек была спроектирована с учётом удобства переноса доказательств из классической математики; в решении, избранном создателями Lean, для этого пришлось пожертвовать достаточно важными свойствами базовой системы CoC, в частности свойством *сохранения типизации*: если $e_1 : T$ и $e_1 \rightarrow e_2$ (« e_1 за один шаг вычисляется в e_2 »), то $e_2 : T$. Повторюсь: это свойство в Lean НЕ выполняется. Если быть точным, можно подобрать типизируемый терм e_1 такой, что результат вычисления e_2 не будет типизироваться. Данное обстоятельство, конечно, не делает данную логическую систему противоречивой (в частности, существует теоретико-модельное доказательство непротиворечивости системы Lean [3]), но является серьёзным неудобством при неумелой работе с системой. Более того, если такое базовое синтаксическое свойство не выполнено, возникают сомнения в применимости подобной системы для программирования, т.е. акта дизайна программ для исполнения на переписывающей системе.

Таким образом, можно сделать вывод, что на данный момент не существует *языка программирования* на основе CoC, сохраняющего важные вычислительные свойства и пригодного для разработки производительных программных систем целиком в этом языке.

Объект и предмет исследования

Объектом данного исследования является система типов CoC и её расширения, а также практическая реализация эффективной среды исполнения CoC.

Предметом исследования являются такие аспекты CoC и её расширений, которые сделают её пригодной для промышленного применения в качестве системы исполнения высокопроизводительных программ, корректных по построению.

³В последнее время стала популярной методика, по которой доказательство математических утверждений ищут с помощью больших языковых моделей, генерирующих программы в одной из таких систем. [10]

⁴В Rosq, однако, есть обходной механизм *извлечения доказательства*, транслирующий скрипт Rosq в программу на OCaml/Haskell/Scheme/...

Цель и задачи исследования

Целью данной работы является поиск системы типов, основанной на CoC, пригодной для написания корректных по построению программ и доступной широкому кругу специалистов в сфере информационных технологий, а также её эффективная реализация.

Для достижения этой цели необходимо выполнить следующие задачи:

1. Исследование возможных (как существующих, так и новых) расширений системы CoC;
2. Отбор наиболее необходимых для мейнстримного программирования расширений;
3. Доказательство сохранения важных вычислительных свойств в расширенной системе;
4. Дизайн эргономичного алгоритма вывода типов, поддерживающего существующие и будущие паттерны программирования с зависимыми типами;
5. Реализация эффективной среды исполнения и логического ядра системы;
6. Создание демонстрационных проектов на новом языке, показывающих состоятельность выбранного подхода.

Методология и методы исследования

В силу природы поставленных задач, настоящее исследование производится на стыке математической логики (теория типов, теория моделей), алгебры (символьные вычисления, алгебраическая оптимизация) и теоретической информатики (теория алгоритмов, теория сложности) с акцентом на практической реализации. Для каждой из поставленных задач используются свои методы исследования, в частности:

1. Полученная система типов описывается в декларативной форме, в виде генценовского исчисления секвенций специального вида;
2. Метатеоретические свойства системы доказываются относительно сформулированного исчисления структурной индукцией по дереву естественного вывода в системе;
3. Алгоритм вывода типов также формулируется с помощью исчисления секвенций, но уже в алгоритмической форме, демонстрируется корректность вывода относительно базовой системы;
4. Реализация компилятора и среды исполнения опирается на новейшие практики реализации подобных программных систем;
5. Эффективность среды исполнения проверяется посредством бенчмаркинга и сравнения производительности по сравнению с аналогичными средами на эталонных программах, реализованных на различных языках.

Научная новизна исследования

Данное исследование предлагает новые дизайны для систем типов с зависимыми типами с акцентом на практическую применимость в промышленном программировании, основываясь на новейших результатах в области оснований математики; кроме этого, настоящая работа содержит значимые продвижения в техниках создания эффективных сред исполнения для языков с зависимыми типами, а также в техниках использования зависимых типов в промышленном программировании.

Практическая значимость работы

Существование удобного в использовании языка программирования с эффективной средой исполнения, позволяющего создавать корректные по построению программы, качественно изменит подход к разработке программного обеспечения. На подобные программные решения существует большой спрос, что показывает пример Rust [1]; технология, позволяющая раздвинуть границы возможного ещё дальше, будет востребована не меньше.

Степень достоверности и апробация результатов

В лучших традициях дисциплины *теории типов*, достоверность ключевых теоретических результатов, полученных в ходе исследования, гарантируется формализацией доказательств в среде интерактивных доказательств Rosq. Апробация практических результатов произведена с помощью реализации ряда демонстрационных проектов в полученном языке программирования, показывающих состоятельность и перспективность выбранного подхода.

1. Резюме представленных к защите публикаций

(Вместо этой главы будет несколько нумерованных глав)

Заключение

Основные результаты исследования

TODO

С акцентом на изложение полученных научных результатов (а не на пересказ содержания диссертации)

Список выносимых на защиту результатов TODO

Общие выводы

TODO

(без повторения основных результатов)

Перспективы дальнейшей разработки

TODO

Основные публикации соискателя

TODO

А. Список сокращений и условных обозначений

В. Словарь терминов

С. Список литературы

- [1] National Security Agency. *Software Memory Safety*. Апр. 2023. URL: https://media.defense.gov/2022/Nov/10/2003112742/-1/-1/0/CSI_SOFTWARE_MEMORY_SAFETY.PDF.
- [2] Thorsten Altenkirch и Nils Anders Danielsson. «Termination Checking in the Presence of Nested Inductive and Coinductive Types». В: *Partiality and Recursion in Interactive Theorem Provers, PAR@ITP 2010, Edinburgh, UK, July 15, 2010*. Под ред. Ekaterina Komendantskaya, Ana Bove и Milad Niqui. Т. 5. EPIС Series. EasyChair, 2010, с. 101—106. DOI: 10.29007/N51D.
- [3] Mario Carneiro. «The Type Theory of Lean». Дис. ... маг. Carnegie Mellon University, апр. 2019.
- [4] Ranald Clouston и др. *Programming and Reasoning with Guarded Recursion for Coinductive Types*. 2015. arXiv: 1501.02925 [cs.PL].
- [5] Thierry Coquand и Gérard Huet. «The calculus of constructions». В: *Information and Computation* 76.2 (1988), с. 95—120. ISSN: 0890-5401. DOI: [https://doi.org/10.1016/0890-5401\(88\)90005-3](https://doi.org/10.1016/0890-5401(88)90005-3).
- [6] Edsger W. Dijkstra. «The humble programmer». В: *Commun. ACM* 15.10 (окт. 1972), с. 859—866. ISSN: 0001-0782. DOI: 10.1145/355604.361591.
- [7] Leonardo de Moura и Sebastian Ullrich. «The Lean 4 Theorem Prover and Programming Language». В: *Automated Deduction – CADE 28*. Под ред. André Platzer и Geoff Sutcliffe. Cham: Springer International Publishing, 2021, с. 625—635. ISBN: 978-3-030-79876-5.
- [8] Chris Newcombe и др. «How Amazon web services uses formal methods». В: *Commun. ACM* 58.4 (март 2015), с. 66—73. ISSN: 0001-0782. DOI: 10.1145/2699417.
- [9] Ulf Norell. «Towards a practical programming language based on dependent type theory». Дис. ... док. SE-412 96 Göteborg, Sweden: Department of Computer Science и Engineering, Chalmers University of Technology, сент. 2007.
- [10] Peiyang Song, Kaiyu Yang и Anima Anandkumar. *Lean Copilot: Large Language Models as Copilots for Theorem Proving in Lean*. 2025. arXiv: 2404.12534 [cs.AI].
- [11] The Coq Development Team. *The Coq Proof Assistant*. Вер. 8.20. Сент. 2024. DOI: 10.5281/zenodo.14542673.
- [12] Andrea Vezzosi, Anders Mörtberg и Andreas Abel. «Cubical agda: a dependently typed programming language with univalence and higher inductive types». В: *Proc. ACM Program. Lang.* 3.ICFP (июль 2019). DOI: 10.1145/3341691.
- [13] Philip Wadler. «Propositions as types». В: *Commun. ACM* 58.12 (нояб. 2015), с. 75—84. ISSN: 0001-0782. DOI: 10.1145/2699407.
- [14] Б. Пирс. *Типы в языках программирования: пер. с англ.* Лямбда пресс, 2012. ISBN: 9785791300829.

D. Список иллюстрированного материала

Е. Приложения